

Software Engineering Lab - Coursework

Lab 4: VibeCoding - LLM Pair Programming & Number Guessing Repair

Student Name	Shakthi Raja G P
SRN & Section	PES1UG25CS843 Section L (Semester 5)
Assigned Lab	Lab 4: Number Guessing Repair (Pygame)
AI / LLM Tool Used	Claude (Anthropic) - Sonnet
Conversation Share Link	https://claude.ai/share/2050e651-0525-499d-b819-7c1b1b4b1b4f

1. Overview & Objectives

The objective of Lab 4 is to use modern Vibe Coding and AI pair-programming tools (Claude / Anthropic) to diagnose a broken Pygame-based number deduction game, fix deliberate runtime bugs safely, and incrementally develop all requested enhancement features in fewer than 3–4 prompting iterations per task. Each task follows an isolated git commit with verification.

2. Task 1: Fix Empty Input Crash Bug

Problem Analysis: Submitting a guess with an empty text box raised `ValueError: invalid literal for int() with base 10: ''` because `int(self.input_box.text)` executed unconditionally without input sanitization.

Prompt Sent to Claude:

I am working on a Pygame Number Guessing Game for a software engineering lab.

Problem Description (Task 1):

When the user clicks SUBMIT or presses Return without typing any digits in the input box, the game crashes immediately with:

```
ValueError: invalid literal for int() with base 10: ''
```

Requirements:

1. Validate whether `self.input_box.text` is non-empty and contains valid digits before converting to `int`.
2. If empty, do not convert, do not increment `self.attempts`, and update `self.feedback_msg` to prompt the player to enter a valid number first (e.g.

```
"Please enter a valid number first!") with an appropriate warning color.  
3. Preserve the attempt count and ensure the game does not crash.
```

Solution Applied & Commit:

Claude provided an updated `submit_guess()` method validating text with `text.strip()` and not `text.isdigit()`. Invalid submissions set a warning message, clear the box, and return early without incrementing attempts.

Commit: b635928 - fix(lab-04): validate input before conversion to prevent empty input crash

3. Task 2: Implement Dynamic Range Hints

Problem Analysis: The game previously only stated whether a guess was too low or high. Students must maintain running boundaries `[low_bound, high_bound]` narrowing the search space on each valid guess.

Prompt Sent to Claude:

```
Enhance game/game_engine.py to:  
1. Maintain self.low_bound = 1 and self.high_bound = 100 in __init__.  
2. Update bounds in submit_guess():  
   - If guess < target: low_bound = max(low_bound, guess + 1)  
   - If guess > target: high_bound = min(high_bound, guess - 1)  
3. Reset low_bound to 1 and high_bound to 100 in reset().  
4. Render the refined range on screen (e.g. "Current Possible Range: 24 - 68").
```

Solution Applied & Commit:

Claude updated the engine bounds tracking and blitted the range surface directly below the feedback text at `y=268`, resetting cleanly on restart.

Commit: b522929 - feat(lab-04): add dynamic range hints narrowing possible bounds on guesses

4. Task 3: Implement Visual Attempt History Log

Problem Analysis: Players only had a raw attempt counter. A visual history of the last 5 guesses with directional tags/arrows aids player deduction.

Prompt Sent to Claude:

```
Implement an attempt history log:  
1. Maintain history of the last 5 guesses with status ('LOW', 'HIGH',  
'CORRECT').  
2. Render visual badge chips centered horizontally with directional colored  
arrows (up for LOW, down for HIGH, checkmark for CORRECT) drawn using pygame  
polygon shapes to prevent font glyph incompatibility.  
3. Reset history on game restart.
```

Solution Applied & Commit:

Claude added `_render_history()` drawing geometric chip containers and arrow polygons at `y=302` without overlapping existing elements.

Commit: 52a01a4 - feat(lab-04): add visual guess history panel with directional arrow indicators

5. Task 4: Maximum Attempts Limit & Game Over State

Problem Analysis: Unlimited guesses removes difficulty. Added a strict maximum allowance of 7 attempts triggering a distinct loss screen.

Prompt Sent to Claude:

```
Implement maximum attempts limit and Game Over state:  
1. Define MAX_ATTEMPTS = 7 and show "Attempts: X / 7" on screen.  
2. Track game_over state in __init__ and reset().  
3. If attempts reach 7 without finding the secret number, trigger game_over =  
True, display "GAME OVER! The number was X." in red, and prompt "Press [R] to  
Try Again".  
4. Allow pressing [R] to restart on both game_won and game_over.
```

Solution Applied & Commit:

Claude finalized the engine state machine with max attempts enforcement, red loss feedback, and universal [R] key restart handling.

Commit: 4794348 - feat(lab-04): add maximum attempts limit and game over state with restart

6. Deliverables & Artifact Verification

All deliverables specified in the course syllabus and repository checklist have been fulfilled:

- video-before.mp4: 10s gameplay video demonstrating the empty input crash (Commit 9db35f2)
- video-after.mp4: 10s gameplay video demonstrating input validation, range hints, history chips, and game over state (Commit 8141b41)
- Full Codebase: Fully tested, robust, and cleanly modularized in lab-04/game/
- Claude Public Chat Share URL: <https://claude.ai/share/2050e651-0525-499d-b819-7c1b1b4b1b4f>
- Documented History: chat-history.docx and chat-history.pdf.